

Close the `_admin` Bootstrap Path — Design

Status: Spec 1 of 2. Shippable independently. **Date:** 2026-04-18 **Related:** Spec 2 — “Email-based recovery & onboarding” (to be written after spec 1 lands).

Problem

`OB2_BRAIN_KEY` is the bootstrap secret for a virtual `_admin` account. It is usable in three places:

1. `POST /auth/login` as `_admin` + brain-key → session cookie.
2. Authorization: Bearer `<brain-key>` against `/admin/*` and every other `bearerAuthMulti`-guarded route.
3. `x-brain-key: <brain-key>` against MCP endpoints.

Once a real global admin exists, every one of those paths is redundant and a standing risk. A leaked brain-key (env file, shell history, log line, shoulder-surf at the login form) is equivalent to global-admin access.

Goal

Once a real, enabled global admin exists in `users.json`, the brain-key stops authenticating anywhere. Reversible: if the real admin is later removed or disabled, the brain-key auto-reopens for re-bootstrap.

Non-goals (deferred to spec 2)

- Email-based password recovery.
- SMTP configuration.
- `email` field on user records.
- Invite-email onboarding.

Solo-admin logout recovery in spec 1 is handled by a shell script, not email.

Core invariant

`hasRealGlobalAdmin()` returns true iff `users.json` contains at least one user where `enabled !== false`, `global_admin === true`, and `username !== "_admin"`.

Every gate and safety rail in this spec is keyed on that single predicate.

Scope

1. Auth gates — close brain-key once real admin exists

Three call sites, one predicate.

`server/routes/auth.ts` — `POST /auth/login` : If `username === "_admin"` :

- If `password !== config.brainKey` → 401 `invalid credentials`.
- Else if `hasRealGlobalAdmin()` → 403 `bootstrap _admin is disabled because a real global admin exists. Sign in as that user instead.`
- Else → mint `_admin` session (existing behavior).

`server/users.ts` — `_resolveAuth` (used by `bearerAuthMulti` and `mcpAuthMulti`): The single-key fallback at line ~108 becomes conditional:

```
if (key === config.brainKey) {  
  if (hasRealGlobalAdmin()) return null;  
  return { username: "_admin", global_admin: true, domains: {} };  
}
```

This closes both the `Authorization: Bearer` path and the `x-brain-key` MCP path simultaneously.

Side-effects on admin create / promote (from existing stash):

- `POST /admin/users` with `global_admin: true` → `revokeUserSessions("_admin")`.
- `PATCH /admin/users/:name` that sets `global_admin: true` → same.

Public status endpoint: `GET /auth/status` → `{ bootstrap_available: boolean }`. Unauthenticated. Used by the login page to render the hint truthfully. Exposes only the boolean; no secrets.

2. Zero-admin safety rail

Applies everywhere `users.json` can be modified through a server API.

Helper in `server/users.ts` :

```
/** True if applying `next` would leave zero enabled, non-_admin global admins.  
 * Used to refuse writes that would lock everyone out. */  
export function wouldLeaveZeroAdmins(next: Map<string, UserRecord>): boolean
```

Applied before commit in:

Path	Current behavior	New behavior
PATCH /admin/users/:name (demote self / disable self)	Succeeds, locks out	409 would leave no global admin
DELETE /admin/users/:name (revoke)	Succeeds, locks out	409 same
POST /admin/users/raw (new — see §3)	N/A	400 same

The rule does **not** block the “promote successor, then demote self” flow — that transitions admin-count $1 \rightarrow 2 \rightarrow 1$ and the intermediate state satisfies the rule.

3. Raw `users.json` editor

UI placement: Users tab, below the existing typed CRUD, inside a collapsed `<details>` block titled `Advanced: edit users.json directly`. Global-admin-only. Collapsed by default. Small muted warning that edits bypass typed validation beyond schema.

Endpoints:

Method	Path	Auth	Request	Response
GET	/admin/users/raw	global admin	—	200 {content: string, path: string, mtime: string (ISO-8601)}
POST	/admin/users/raw	global admin	{content: string, expected_mtime: string (ISO-8601)}	200 {ok:true, mtime: string} / 400 / 409

View behavior: return `users.json` verbatim. `password_hash` (argon2id) and `api_key_hash` (SHA-256) are already one-way, and typed endpoints show the same structural data to a global admin. Masking would require a round-trip merge layer that is easy to get wrong and delivers little.

Save pipeline:

1. Parse body.content as JSON. Parse errors → 400 with line and column.
2. Validate schema: each record has non-empty `username`; `domains` values in `{"read", "write", "admin"}`; types correct. Schema errors → 400 with field path.
3. Call `wouldLeaveZeroAdmins(nextState)`. If true → 400 with the canonical message.
4. Concurrency: compare `body.expected_mtime` to current file mtime. Mismatch → 409 `users.json` was modified by someone else — reload and retry.
5. Atomic write: write `users.json.tmp`, `fsync`, `rename` over `users.json`.
6. `_reloadIfChanged()` picks up the new state in the running server.
7. For each user whose record materially changed (disabled, demoted, `password_hash` changed) → `revokeUserSessions(username)`.

4. Shell break-glass script

Path: `server/scripts/reset-admin.ts` . **Invocation:** `deno run --allow-read --allow-write server/scripts/reset-admin.ts <username> [--password <pw>] [--promote]` , typically through `docker exec` .

Behavior:

- Resolves `users.json` path the same way the server does (config file + env override).
- If `--password` omitted, reads from stdin with echo disabled.
- Validates password against `validatePasswordStrength` (shared with server).
- Hashes with the same argon2id parameters the server uses (reuses `server/auth/passwords.ts`).
- `--promote` flag: if the user does not exist, create them with `{domains:{}, global_admin:true, enabled:true}` and the provided password. If they exist, set `global_admin:true` and `enabled:true` .
- Atomic write via tmp + rename.
- Exits nonzero with a specific error message on any failure.

Why a separate script, not a CLI subcommand of the main server: it runs when the container is crash-looping. It has no port bindings, no server state, no flag plumbing. Single file, single responsibility.

5. Documentation updates

`docs/user-guide.md` :

- Replace the “Making `_admin` unreachable” troubleshooting section (“WIP, ask the maintainer”) with the new automatic behavior: closes automatically once a real global admin exists; brain-key auto-reopens if the real admin is later removed/disabled.
- Add “If you lock yourself out” sub-section linking to the shell script (§4) with a copy-pasteable `docker exec` example.
- Add a short paragraph under the Users tab section describing the raw editor and its limitations (schema validation only, no round-trip semantic checks).

`README.md` : no change needed (doc index already points to user-guide).

6. Tests

Additions to `tests/e2e.sh` :

- After real admin creation: `_admin` + brain-key dashboard login → 403.
- After real admin creation: `Authorization: Bearer <brain-key>` on `/admin/domains` → 401.
- After real admin creation: `x-brain-key: <brain-key>` on MCP endpoint → 401.
- `DELETE /admin/users/<last-global-admin>` → 409.
- `PATCH /admin/users/<last-global-admin>` with `global_admin: false` → 409.
- `POST /admin/users/raw` with payload that removes last admin → 400.
- `POST /admin/users/raw` with stale `expected_mtime` → 409.
- `POST /admin/users/raw` happy path: load → edit a domain permission → save → verify via `GET /admin/users`.
- Shell script: create a real admin with the script, then log in through `/auth/login` with the new password.
- Brain-key re-opens: remove the real admin from `users.json` on disk → `/auth/status` flips to `bootstrap_available: true` → `_admin` + brain-key login succeeds.

Target pass count: 30+ assertions. Updates to `docs/test-results.md` once implementation lands.

Data model

No changes. `users.json` schema is unchanged. No new on-disk files. No new config values. Spec 2 will add an `email` field and SMTP configuration.

Error handling

- 400 for client-correctable errors (bad JSON, schema violation, zero-admin).
- 401 for bad credentials (brain-key no longer resolves).
- 403 for correct credentials against a disabled bootstrap path.
- 409 for stale-mtime conflicts on raw save.
- 500 only for unexpected server errors (write failure, disk full).

All error responses are `application/json` with an `error` field containing a human-readable message.

Rollback

Every change is code + docs. Revert the commit → brain-key works everywhere again. `users.json` schema is unchanged so there is no data migration to undo.

Risks & mitigations

Risk	Mitigation
Admin demotes/disables themselves as only admin	Zero-admin rail (\$2) returns 409
Admin pastes broken JSON into raw editor	Parse + schema validation before write (\$3)
Two admins edit raw simultaneously	<code>expected_mtime</code> 409 conflict (\$3)
Admin removes <code>password_hash</code> for themselves via raw	Allowed. Recovery paths: another global admin resets via <code>POST /admin/users/:name/password</code> , or shell script (\$4). Session revoked on save so state is consistent.
Brain-key environment still leaks after real admin exists	Not a problem — no longer authenticates anything
Solo admin loses password	Shell script (\$4) via <code>docker exec</code> ; documented in user guide
<code>users.json</code> file deleted / corrupted on disk	<code>hasRealGlobalAdmin()</code> returns false → brain-key auto-reopens → re-bootstrap

Open questions

None blocking. Spec 2 will address email-based recovery and onboarding.